NLD
NONLINEAR DYNAMICS LABORATORY

When amplitude changes the story

In nonlinear structures, amplitude changes frequency, mode shape and stability.



USER MANUAL

NCVD toolbox

User manual for the Nonlinear Characteristic Value Decomposition in Python and MATLAB, version 1.0

Nonlinear Dynamics Laboratory

October 2026

Contents

1 Overview **3**

1.1 Quick start . . . 4

2 The decomposition **5**

2.1 Data and notation . . . 5

2.2 Algorithm . . . 5

2.3 Two propositions . . . 5

2.4 Geometry . . . 6

2.5 Conventions . . . 7

3 Practical data **7**

3.1 Rates from displacement . . . 9

3.2 Sensor noise . . . 9

3.3 Measured forcing . . . 10

3.4 Ensembles . . . 10

3.5 Time units, conditioning and rank . . . 10

4 Nonlinear systems **11**

4.1 Sliding windows . . . 11

4.2 Requirements and caveats . . . 12

4.3 Reduced-order-model diagnostics . . . 13

5 Reference **13**

5.1 Installation . . . 13

5.2 Functions . . . 13

5.3 Options of `ncvd` / `decompose` . . . 14

5.4 Result fields . . . 15

6 Validation **15**

7 Limitations **15**

1 Overview

The NCVD toolbox identifies characteristic values (poles) and characteristic shapes from a measured or simulated state record. For a record with n stations and m samples it writes

$$\mathbf{X}_1 = [\mathbf{q},\ \dot{\mathbf{q}}] = \mathbf{U}\boldsymbol{\Phi}^{\mathsf{H}}, \qquad \mathbf{X}_2 = [\dot{\mathbf{q}},\ \ddot{\mathbf{q}}] = \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Phi}^{\mathsf{H}} + \mathbf{R}, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda_i), \tag{1}$$

where $\mathbf{U}$ is the temporal basis, $\boldsymbol{\Phi}$ the characteristic shapes, λ_i the characteristic values and $\mathbf{R}$ everything the diagonal form cannot represent. The first identity is exact. The second is exact for a noise-free linear record. For noisy, forced or nonlinear data, $\mathbf{R}$ is the least-squares remainder.

The decomposition descends from the characteristic value decomposition (CVD) and its generalisation (GCVD) of Li, Stein and Chelidze [1]. It keeps their data model and quotient but changes how values and shapes are read out, and it adds the tools a practical identification needs.

What is new relative to the published GCVD.

- **Exact readout.** The values are the eigenvalues of $\boldsymbol{\Sigma}_2$, not its diagonal, and the shapes are rotated accordingly. Both identities in Eq. (1) hold exactly, and λ_i belongs to column i of $\boldsymbol{\Phi}$ by construction (Section 2.2).
- **Exactness conditions.** Proposition 2 states when the published diagonal readout is exact and how large its error is when it is not (Section 2.3).
- **Time-unit control.** The balanced quotient applies the Joukowski map $\lambda \mapsto \frac{1}{2}(\lambda + \lambda^{-1})$, which depends on the unit of time. `time_scale='auto'` places every pole where that map is one-to-one (Section 3.5).
- **Rates from displacement.** Any FIR derivative stencil, including Savitzky–Golay, with exact pole correction (Section 3.1).
- **Sensor noise.** Least squares (canonical), bias-compensated least squares and total least squares (Section 3.2).
- **Measured forcing**, **ensembles** of records with different sampling rates, **amplitude tracking** for nonlinear systems, and **reduced-order-model diagnostics** (Sections 3.3–4.3).

Where to get it. Code, manual, slides and animations: `github.com/nld-uri/ncvd-toolbox` (MIT license). Overview page: `web.uri.edu/nld/ncvd-toolbox`.

Contents of the toolbox.

Folder	Contents
`python/ncvd/`	Python package (NumPy, SciPy; Matplotlib for figures)
`python/examples/`	seven worked examples and the paper replication
`python/tests/`	28 unit tests, reference copy of the original `ncvd.py`
`matlab/+ncvd/`	MATLAB package, base MATLAB R2022a or later, no toolboxes
`matlab/examples/`, `matlab/tests/`	the same examples; 13 tests including Python cross-checks
`docs/`	this manual, the slide deck and the animations

1.1 Quick start

Python:

```python
import numpy as np
import ncvd as nc                                   # after: pip install -e python
from ncvd import systems as sy

M, C, K = sy.paper_chain10()                        # 10-DOF chain of [1], App. C.1
t = np.arange(0, 60, 0.05)
d, v, a = sy.simulate_linear(M, C, K, np.random.randn(20), t)

rec = nc.ncvd(d, v, a)                              # X1 = U Phi^H, X2 = U Lam Phi^H +
    R
mp  = nc.modal_parameters(rec.lam)                  # wn, wd, zeta, fn, fd
idx = nc.physical_modes(rec.lam)                    # one pole per conjugate pair
shapes = rec.Psi[:10, idx]                          # displacement part of each mode
print(rec.res, rec.exact1)                          # both ~1e-15 for this record
```

MATLAB:

```matlab
addpath('path/to/NCVD Toolbox/matlab')
[M, C, K] = ncvd.paperChain10();
t = (0:0.05:60-0.05)';
[d, v, a] = ncvd.simulateLinear(M, C, K, randn(20,1), t);
rec = ncvd.decompose(d, v, a);
mp  = ncvd.modalParameters(rec.lam);
idx = ncvd.physicalModes(rec.lam);
shapes = rec.Psi(1:10, idx);
```

Choosing options.

Your data	Use
Displacement, velocity, acceleration measured or simulated	`ncvd(d, v, a)`
Displacement only	`state_from_displacement`, then `correct_poles` (Sec. 3.1)
White sensor noise, sampling well above the highest mode	`operator='bcls', noise_var='auto', fs=fs`
Measured excitation	`forcing=F`
Several records, any sampling rates	`ncvd_ensemble(records)`
Spectrum spanning decades	`rank=r` (whitening)
Lightly damped, nondimensional model ($\omega \approx 1$)	`time_scale='auto'`
Nonlinear free decay	`track(d, v, a, fs=..., length=..., step=...)`
Multiple equilibria	`center=True`

2 The decomposition

2.1 Data and notation

$\mathbf{Y}_1, \mathbf{Y}_2 \in \mathbb{C}^{m\times p}$ are two records with time along the rows. For a state record $\mathbf{Y}_1 = \mathbf{X}_1 = [\mathbf{q}, \dot{\mathbf{q}}]$ and $\mathbf{Y}_2 = \mathbf{X}_2 = [\dot{\mathbf{q}}, \ddot{\mathbf{q}}]$, so $p = 2n$. For a linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ the rows satisfy $\mathbf{X}_2 = \mathbf{X}_1\mathbf{A}^{\mathsf{T}}$. The second-moment blocks are $\mathbf{K}_{ij} = \mathbf{Y}_i^{\mathsf{H}}\mathbf{Y}_j$, and $\mathbf{B} = \mathbf{Y}_1^{\dagger}\mathbf{Y}_2$ is the least-squares operator of $\mathbf{Y}_2 \approx \mathbf{Y}_1\mathbf{B}$. `ncvd_pair(Y1, Y2)` accepts any pair, for example shift-map or Hankel data; `ncvd(d, v, a)` builds the state pair.

2.2 Algorithm

1. **Preprocess** (optional): remove column means, apply row weights, whiten $\mathbf{Y}_1$ to rank r, project a measured forcing out of both blocks, rescale the time unit used in step 2.
2. **Balanced quotient.** $\mathbf{Q} = \frac{1}{4}(\mathbf{K}_{21}\mathbf{K}_{11}^{-1} + \mathbf{K}_{22}\mathbf{K}_{12}^{-1} + \mathbf{K}_{11}\mathbf{K}_{21}^{-1} + \mathbf{K}_{12}\mathbf{K}_{22}^{-1})$, eigenvectors $\mathbf{V}$.
3. **Coordinates and coupling.** $\mathbf{\Xi} = \mathbf{Y}_1\mathbf{V}^{-\mathsf{H}}$ (so $\mathbf{\Sigma}_1 = \mathbf{I}$) and $\mathbf{\Sigma}_2 = \mathbf{V}^{\mathsf{H}}\mathbf{B}\mathbf{V}^{-\mathsf{H}}$.
4. **Rotation.** $\mathbf{\Sigma}_2 = \mathbf{T}\mathbf{\Lambda}\mathbf{T}^{-1}$; $\mathbf{U} = \mathbf{\Xi}\mathbf{T}$, $\mathbf{\Phi} = \mathbf{V}\mathbf{T}^{-\mathsf{H}}$; $\mathbf{R} = \mathbf{Y}_2 - \mathbf{U}\mathbf{\Lambda}\mathbf{\Phi}^{\mathsf{H}}$.

The published GCVD stops after step 3. It returns $\mathbf{V}$ as the shapes and the diagonal of $\mathbf{\Sigma}_2$ as the values [1, Algorithm 1, Eqs. (13)–(14)]. Step 4 is the change.

2.3 Two propositions

Proposition 1 (Canonical values are least-squares values). *Let* $\mathbf{Q}$ *and* $\mathbf{\Sigma}_2$ *be diagonalisable and* $\mathbf{E} = \mathbf{V}^{-\mathsf{H}}\mathbf{T}$. *Then* $\mathbf{B}\mathbf{E} = \mathbf{E}\mathbf{\Lambda}$, $\mathbf{U} = \mathbf{Y}_1\mathbf{E}$ *and* $\mathbf{\Phi} = \mathbf{E}^{-\mathsf{H}}$ *for whichever operator* $\mathbf{B}$ *is used. Hence* λ_i *are the eigenvalues of* $\mathbf{B}$, *and the column spaces of* $\mathbf{U}$ *and* $\mathbf{\Phi}$ *do not depend on* $\mathbf{Q}$; $\mathbf{Q}$ *fixes the normalisation of*

the columns and the conditioning of the intermediate steps. For the default `operator='ls'` *without forcing,* $\mathbf{B} = \mathbf{Y}_1^{\dagger}\mathbf{Y}_2$*, so the values are least-squares values and* $\mathbf{R} = (\mathbf{I} - \mathbf{Y}_1\mathbf{Y}_1^{\dagger})\mathbf{Y}_2$ *(in whitened or weighted coordinates when* `rank` *or* `weights` *is used).*

Proof. $\mathbf{BE} = \mathbf{BV}^{-\mathsf{H}}\mathbf{T} = \mathbf{V}^{-\mathsf{H}}\mathbf{\Sigma}_2\mathbf{T} = \mathbf{V}^{-\mathsf{H}}\mathbf{T}\mathbf{\Lambda} = \mathbf{E}\mathbf{\Lambda}$; $\mathbf{U} = \mathbf{Y}_1\mathbf{V}^{-\mathsf{H}}\mathbf{T} = \mathbf{Y}_1\mathbf{E}$; $\mathbf{\Phi} = \mathbf{V}\mathbf{T}^{-\mathsf{H}} = (\mathbf{T}^{-1}\mathbf{V}^{\mathsf{H}})^{\mathsf{H}} = (\mathbf{E}^{-1})^{\mathsf{H}}$. Finally $\mathbf{U}\mathbf{\Lambda}\mathbf{\Phi}^{\mathsf{H}} = \mathbf{Y}_1\mathbf{E}\mathbf{\Lambda}\mathbf{E}^{-1} = \mathbf{Y}_1\mathbf{B}$. □

For a state record this is the state-variable decomposition with a least-squares rate model [3], which [1] already identifies as a special case of CVD. The value of the NCVD formulation lies in the exact readout, the diagnostics, and the tools of Sections 3–4, not in a new pole estimator.

Proposition 2 (When the published readout is exact). *The published readout (values* $r_i = (\mathbf{\Sigma}_2)_{ii}$*, shapes* $\mathbf{V}$*) returns the eigenvalues of* $\mathbf{B}$ *together with their eigenvector shapes if and only if* $\mathbf{\Sigma}_2$ *is diagonal, that is, if and only if* $\mathbf{V}$ *are eigenvectors of* $\mathbf{B}^{\mathsf{H}}$*. (A triangular* $\mathbf{\Sigma}_2$ *would still give the right values, but not the right shapes.) In particular, with* $\mathbf{B}$ *the least-squares operator: (i) for the CVD quotient* $\mathbf{K}_{21}\mathbf{K}_{11}^{-1} = \mathbf{B}^{\mathsf{H}}$ *and the "forced" quotient* $\frac{1}{2}(\mathbf{K}_{11}\mathbf{K}_{21}^{-1} + \mathbf{K}_{21}\mathbf{K}_{11}^{-1}) = \frac{1}{2}(\mathbf{B}^{\mathsf{H}} + \mathbf{B}^{-\mathsf{H}})$ *it is exact for any data, provided the map separates the eigenvalues; (ii) for the balanced quotient it is exact when* $\mathbf{Y}_2 = \mathbf{Y}_1\mathbf{B}$ *holds exactly; (iii) for averaged ensemble quotients and for nonlinear windows it is not exact. When* $\mathbf{\Sigma}_2 = \mathbf{D} + \mathbf{F}$ *with* $\mathbf{F}$ *off-diagonal, the diagonal readout's pole error is* $O(\|\mathbf{F}\|^2/\delta)$ *and its shape error is* $O(\|\mathbf{F}\|/\delta)$*, where* δ *is the eigenvalue gap (first-order perturbation theory; fitted slopes 1.74 and 1.05 in Example 7).*

Proof. If $\mathbf{B}^{\mathsf{H}}\mathbf{V} = \mathbf{V}\mathbf{D}$ then $\mathbf{V}^{\mathsf{H}}\mathbf{B} = \mathbf{D}^{\mathsf{H}}\mathbf{V}^{\mathsf{H}}$ and $\mathbf{\Sigma}_2 = \mathbf{D}^{\mathsf{H}}$, which is diagonal; conversely, a diagonal $\mathbf{\Sigma}_2$ means $\mathbf{B}\mathbf{V}^{-\mathsf{H}} = \mathbf{V}^{-\mathsf{H}}\mathbf{\Sigma}_2$. For (i), $\mathbf{K}_{21}\mathbf{K}_{11}^{-1} = (\mathbf{K}_{11}^{-1}\mathbf{K}_{12})^{\mathsf{H}} = \mathbf{B}^{\mathsf{H}}$, so both quotients are functions $f(\mathbf{B}^{\mathsf{H}})$; when f separates the eigenvalues, their eigenvectors are those of $\mathbf{B}^{\mathsf{H}}$. For (ii), $\mathbf{K}_{12}\mathbf{K}_{22}^{-1} = \mathbf{B}_b^{\mathsf{H}}$ with $\mathbf{B}_b = \mathbf{Y}_2^{\dagger}\mathbf{Y}_1$, and $\mathbf{B}_b = \mathbf{B}^{-1}$ exactly when $\mathbf{Y}_2 = \mathbf{Y}_1\mathbf{B}$. The orders follow from first-order perturbation of a diagonal matrix by an off-diagonal one, which leaves the diagonal unchanged. □

`python/examples/paper_replication.py` reruns every experiment of [1] with both readouts. The published results do not change: the experiments there satisfy (i) or (ii). Figure 1 shows both failure modes.

2.4 Geometry

Three pictures carry most of the intuition. Each has an animated version in `docs/animations/` and in the slide deck.

The residual is a projection. Each column of $\mathbf{X}_2$ is split into its orthogonal projection on the column space of $\mathbf{X}_1$, which the diagonal form reproduces exactly, and a remainder orthogonal to it (Figure 2a). For a noise-free linear record the remainder is zero. For a window of a nonlinear decay, $\|\mathbf{R}\|/\|\mathbf{X}_2\|$ measures the part of the rate data that no linear model of that window explains.

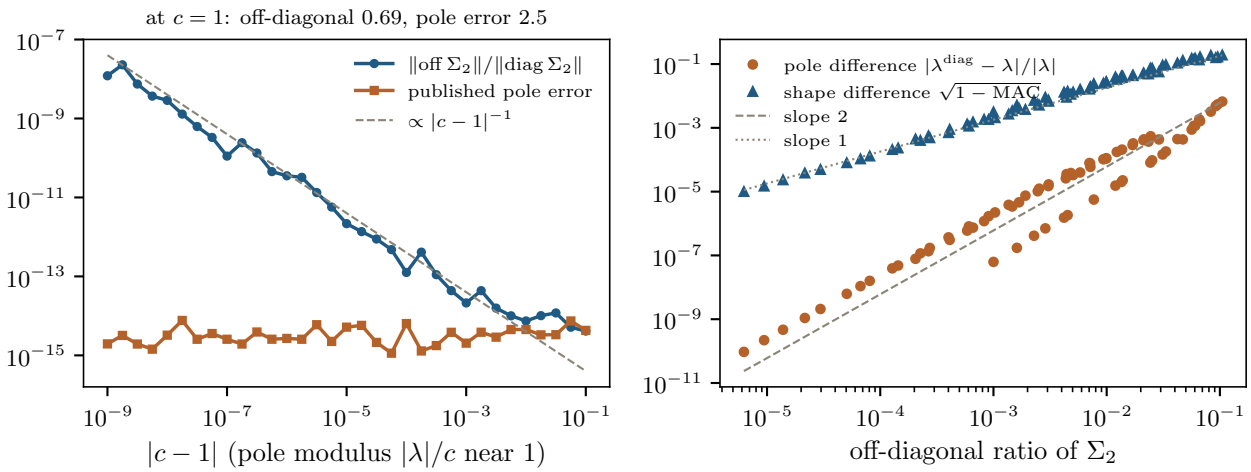


Figure 1 Published against canonical readout. (a) A lightly damped chain with $\omega = 1$, time unit scaled by c; at $c = 1$ a pole meets its conjugate under the balanced map. (b) Windows of a nonlinear decay: pole difference of second order, shape difference of first order in the off-diagonal ratio of Σ_2 (fitted slopes 1.74 and 1.05). Example 7.

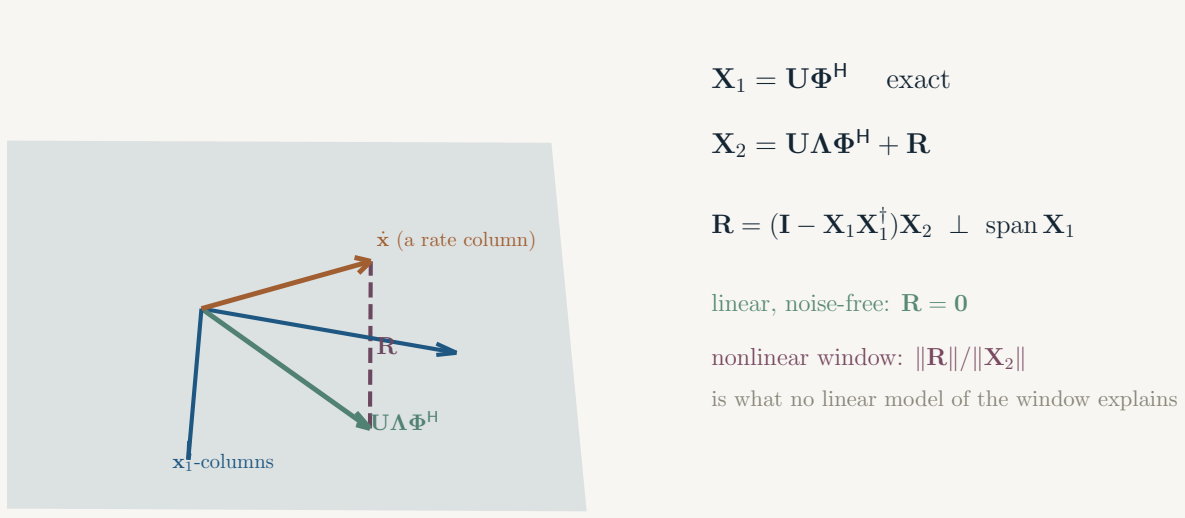
A complex mode is a rotating, shrinking polygon. With non-proportional damping the eigenvector ψ has entries with different phases. The modal contribution $\psi_j e^{\lambda t}$ of each station is a phasor; together they form a polygon that rotates at Im λ and shrinks at Re λ. The motion is its real part, and the nodes travel. The modal phase collinearity (MPC) [1] measures how close the polygon is to a line (Figure 2b).

The balanced quotient folds the plane. For exact linear data, $\mathbf{Q} = \frac{1}{2}(\mathbf{A}+\mathbf{A}^{-1})$, so its eigenvalues are $\mu = \frac{1}{2}(\lambda + \lambda^{-1})$. This Joukowski map is one-to-one outside (or inside) the unit circle, but it sends λ and $1/\lambda$ to the same point and squeezes the unit circle onto $[-1, 1]$. A pole with $|\lambda| = 1$ in the chosen time unit meets its conjugate (Figure 2c). Rescaling time moves the poles relative to the circle; see Section 3.5.

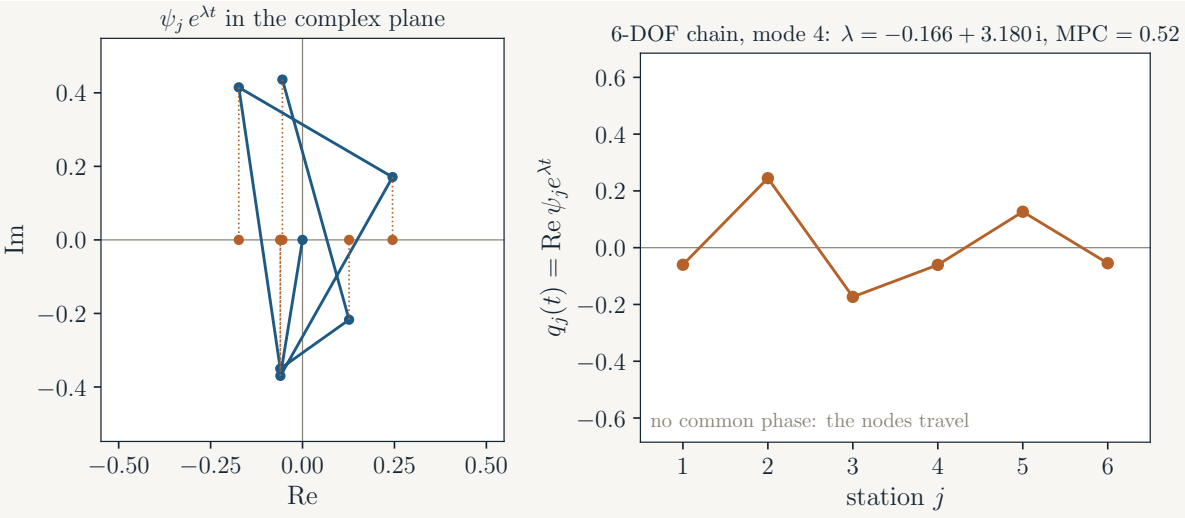
2.5 Conventions

- **$\mathbf{\Phi}$ and $\mathbf{\Psi}$.** $\mathbf{X}_1 = \mathbf{U}\mathbf{\Phi}^{\mathsf{H}}$ gives $\mathbf{x}(t) = \sum_i u_i(t)\,\overline{\phi_i}$. The eigenvector of $\mathbf{A}$ for λ_i is therefore $\psi_i = \overline{\phi_i}$, returned as `Psi`. Compare shapes with `Psi`; its velocity half equals λ_i times its displacement half for a linear record.
- **Order.** By default, modes are sorted by ascending $|\text{Im}\,\lambda|$, positive imaginary part first in each pair, real poles last. `physical_modes` returns one index per pair.
- **Units.** λ is in rad per time unit of the data. `modal_parameters` returns $\omega_n = |\lambda|$, $\omega_d = |\text{Im}\,\lambda|$, $\zeta = -\text{Re}\,\lambda/|\lambda|$, and the same in cycles.

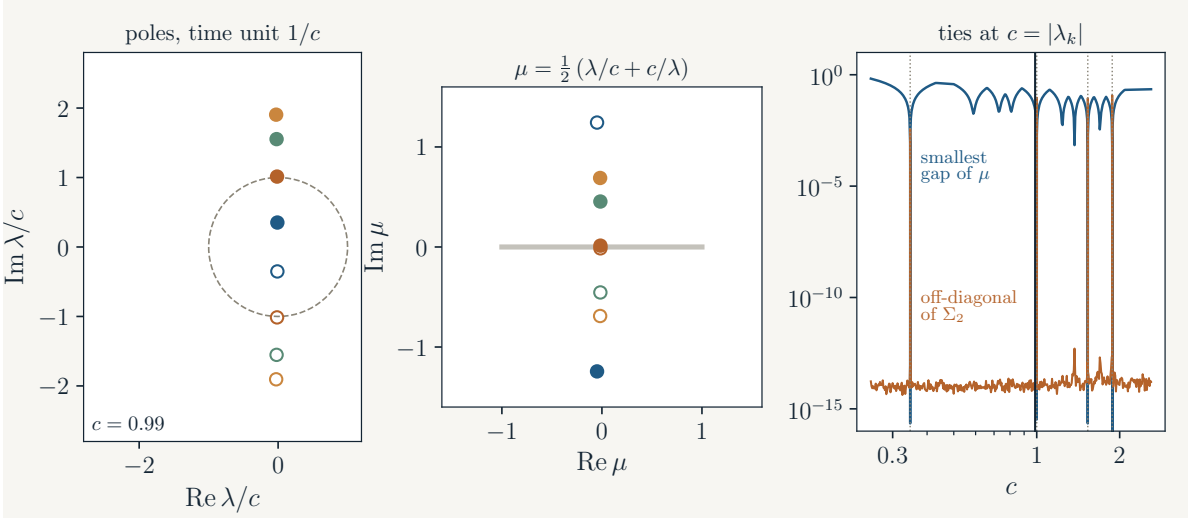
3 Practical data



(a) The fit is the projection of each rate column on span $\mathbf{X}_1$; $\mathbf{R}$ is orthogonal to it.



(b) Mode 4 of a 6-DOF chain with dampers at the first two masses (MPC $= 0.52$).



(c) Poles λ/c, their images μ, and the minimum gap of μ and the off-diagonal ratio of $\mathbf{\Sigma}_2$ against c for a 4-DOF chain with $\omega_2 = 1$.

Figure 2 Three geometric pictures (stills of the animations).

3.1 Rates from displacement

If velocity and acceleration are produced by one linear, shift-invariant FIR stencil D, with $(D\mathbf{q})_k = f_s \sum_j c_j \mathbf{q}_{k+j}$, applied consistently, $\mathbf{X}_1 = [\mathbf{q}, D\mathbf{q}]$ and $\mathbf{X}_2 = [D\mathbf{q}, DD\mathbf{q}]$, then a free (unforced) linear record still satisfies $\mathbf{X}_2 = \mathbf{X}_1\tilde{\mathbf{A}}^\mathsf{T}$ exactly. Here $\tilde{\mathbf{A}}$ has the eigenvectors $[\psi;\, d(\theta)\psi]$, whose displacement half ψ is that of $\mathbf{A}$, and the eigenvalues

$$\tilde{\lambda} = d(\theta), \qquad \theta = e^{\lambda \Delta t}, \qquad d(\theta) = f_s \sum_j c_j \theta^j. \tag{2}$$

The NCVD therefore returns $d(\theta)$. `correct_poles` solves $d(\theta) = \tilde{\lambda}$, keeps the root with the smallest $|\log\theta|$, and returns $f_s \log\theta$. For the forward difference this is Eq. (48) of [1]. The displacement half of each shape is unaffected.

The inversion is unique only where the stencil's symbol is one-to-one along the unit circle. Central-type stencils turn over: $\omega\Delta t$ and $\pi - \omega\Delta t$ give the same central difference. `band()` (MATLAB `filterBand`) returns the limit.

Stencil	`method`	Band $\omega\Delta t <$	Error in band (Ex. 2)
forward / backward	`'forward'`, `'backward'`	π	4.4×10^{-14}
central	`'central'`	$\pi/2$	7.0×10^{-14}
five-point central	`'central4'`	1.798	7.2×10^{-13}
Savitzky–Golay (7, 3) [4]	`'savgol'`	1.141	3.0×10^{-13}

```
d, v, a, filt, k0 = nc.state_from_displacement(q, fs, 'savgol', window=7, polyorder
    =3)
rec = nc.ncvd(d, v, a)
lam = nc.correct_poles(rec.lam, filt, fs)        # then re-sort if needed
assert np.all(np.abs(lam.imag)/fs < filt.band())
```

3.2 Sensor noise

By Proposition 1, the canonical values are least-squares values. White noise in the regressors $\mathbf{Y}_1$ adds $m\,\mathbf{N}$ to $\mathbf{K}_{11}$, with $\mathbf{N}$ the noise covariance, and biases $\mathbf{B}$ toward zero. This is the classical errors-in-variables bias [6]. Three operators are available:

- `'ls'` (default): $\mathbf{B} = \mathbf{Y}_1^\dagger \mathbf{Y}_2$. Exact for clean data, consistent for noise in $\mathbf{Y}_2$ only, and for unmeasured forcing uncorrelated with the present state.
- `'bcls'`: $\mathbf{B} = (\mathbf{K}_{11} - m\mathbf{N})^{-1}\mathbf{K}_{12}$, with only the acceleration block regressed so that $\dot{\mathbf{q}} = \dot{\mathbf{q}}$ stays exact. `noise_var='auto'` estimates $\mathbf{N}$ per channel from the flat spectrum between $0.40 f_s$ and $0.49 f_s$. This needs a sampling rate well above the highest mode.
- `'tls'`: total least squares [5] on column-equilibrated data. Use it only for free decays with measurement noise. Under unmeasured forcing it treats the force as noise, and it fails (Figure 4).

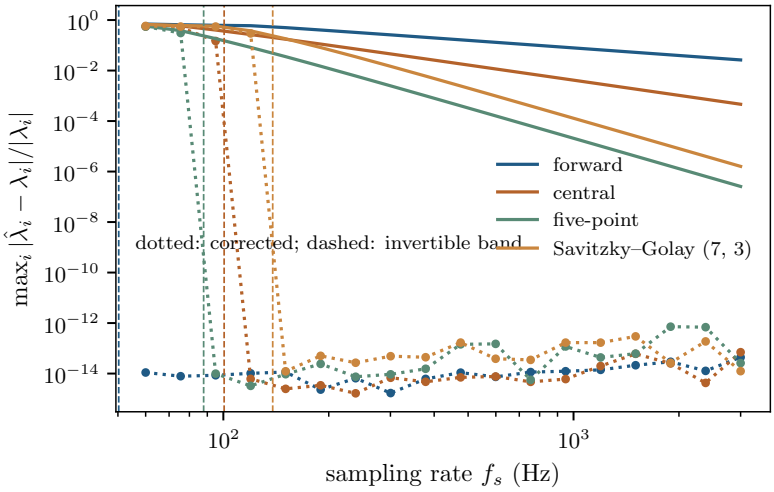


Figure 3 Example 2: 3-DOF chain of [1, App. C.2] (4.24, 70.7 and 158 rad/s). Solid: raw poles from FIR rates. Dotted: after `correct_poles`, at machine precision to the left of each stencil's band limit (dashed).

3.3 Measured forcing

With a measured input record $\mathbf{F}$ ($m \times q$), `forcing=F` projects $\mathbf{F}$ out of both blocks before the operator is estimated (Frisch–Waugh), and returns the input gains as `G`. No mass matrix or input matrix is needed. On the 4-DOF chain of Example 6, driven at three of four masses, the pole error is 2.7×10^{-15} with the forcing and 4.8×10^{-2} without. This replaces Eq. (41) of [1], which requires the forcing in state space and therefore the mass matrix.

For unmeasured random forcing (operational modal analysis), the default operator is the forward regression of $\mathbf{X}_2$ on $\mathbf{X}_1$. It does not use $\mathbf{K}_{22}$, whose forcing autocovariance biases the four-term balanced quotient [1, Sec. 5.3].

3.4 Ensembles

The state-rate relation is continuous-time, so records with different sampling rates, lengths and noise levels share one operator. `ncvd_ensemble` stacks their rows and decomposes once; this is the least-squares fusion of all records. Weights are `'equal'` (default, every record counts equally), `'samples'`, `'snr'` (weight $1/(m_i\,\hat{\sigma}_i^2)$ with $\hat{\sigma}_i^2$ the relative residual variance of record i), or a vector. `rec.blocks[i]` gives each record's rows of $\mathbf{U}$. For the 16 trials of Table 5 of [1] (Example 4), the mean frequency error falls from 1.5 % (one trial) to 0.34 % (16), and the shape error from 0.23 to 0.074.

3.5 Time units, conditioning and rank

The values do not depend on the time unit, but the balanced quotient does (Section 2.4). `time_scale=c` measures time in units of $1/c$ while forming $\mathbf{Q}$; `'auto'` chooses $c = \min|\lambda_{\mathrm{LS}}|/2$ so that every pole has modulus at least 2, where the Joukowski map is one-to-one. With the canonical readout the values are exact either way. The option makes $\mathbf{V}$ nearly

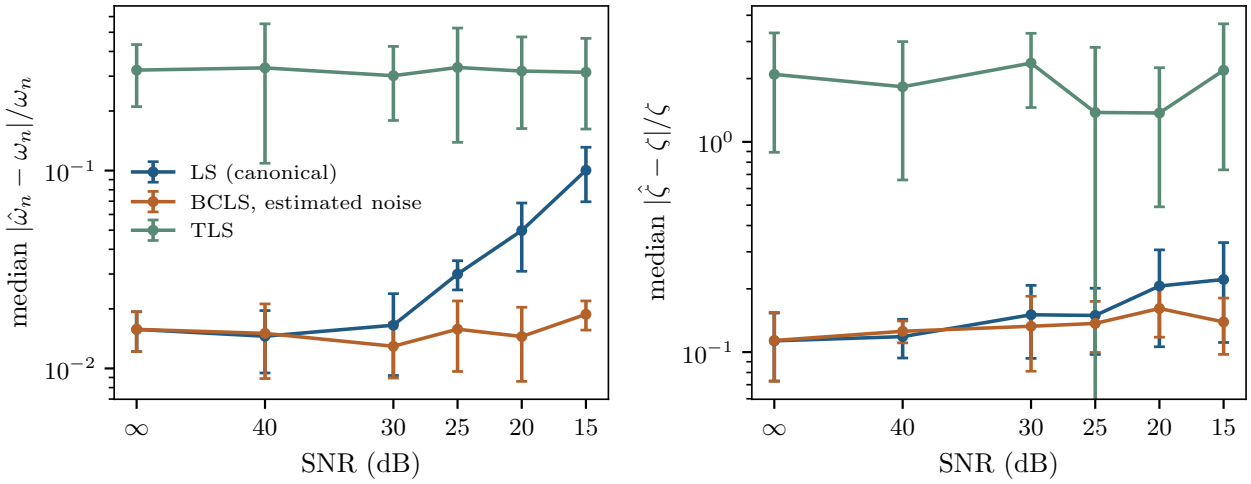


Figure 4 Example 3: the 10-DOF chain under unmeasured white forcing at every mass, 200 Hz for 150 s, independent noise on every channel; mean and spread over 10 records. From ∞ to 15 dB the median frequency error grows from 1.6 % to 10 % with `'ls'` and stays below 1.9 % with `'bcls'`. The remaining damping error (about 11–16 %) is the statistical error of a finite record under random forcing.

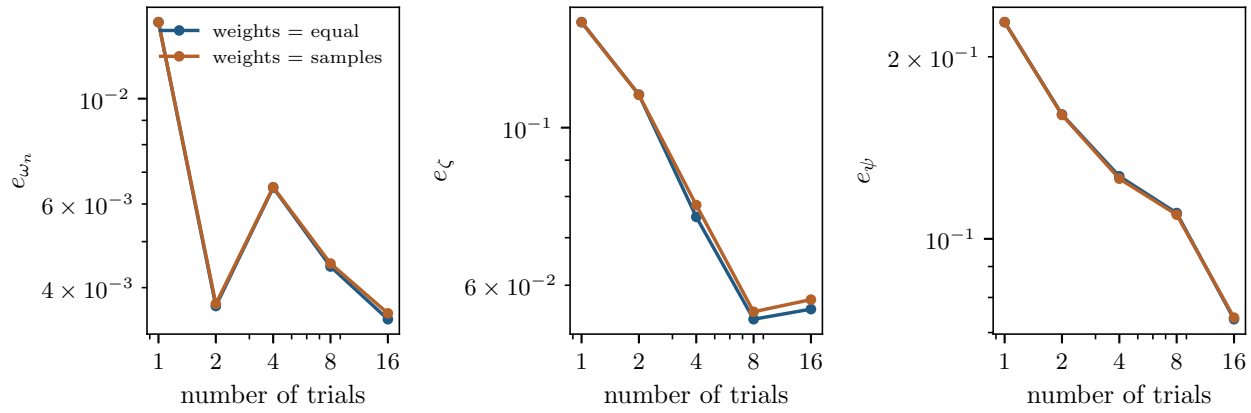


Figure 5 Example 4: ensemble errors against the number of trials.

diagonalise $\mathbf{\Sigma}_2$ (`condT` close to 1), which keeps the intermediate steps well conditioned and makes the published and canonical readouts agree. `track` uses `'auto'` by default.

`rank=r` whitens $\mathbf{Y}_1$ by its SVD before decomposing. It does not change the values when $\mathbf{Y}_1$ has full column rank. It repairs conditioning when the spectrum spans decades or when stations are redundant. Shapes are mapped back to the original coordinates.

4 Nonlinear systems

4.1 Sliding windows

A nonlinear free decay passes through a sequence of energy levels. Each short window is decomposed. Its values are the best linear description of the local dynamics, and following each

resonance through the windows traces its frequency-energy dependence: the data-driven counterpart of a nonlinear normal mode backbone curve [2].

```
tr = nc.track(d, v, a, fs=fs, length=800, step=400, n_track=3, M=M)
tr.wd, tr.zeta, tr.energy, tr.shape, tr.mpc, tr.res      # per window and mode
f1 = [nc.band_frequency(d[s:e, 0], 1/fs, 0.6, 1.1) for s, e in tr.edges]
```

Consecutive windows are matched by minimising $(1-\mathrm{MAC})+|\log(\omega/\omega_{\mathrm{prev}})|$ as an assignment problem. With a mass matrix, `amp` and `energy` are mass-normalised modal amplitude and energy.

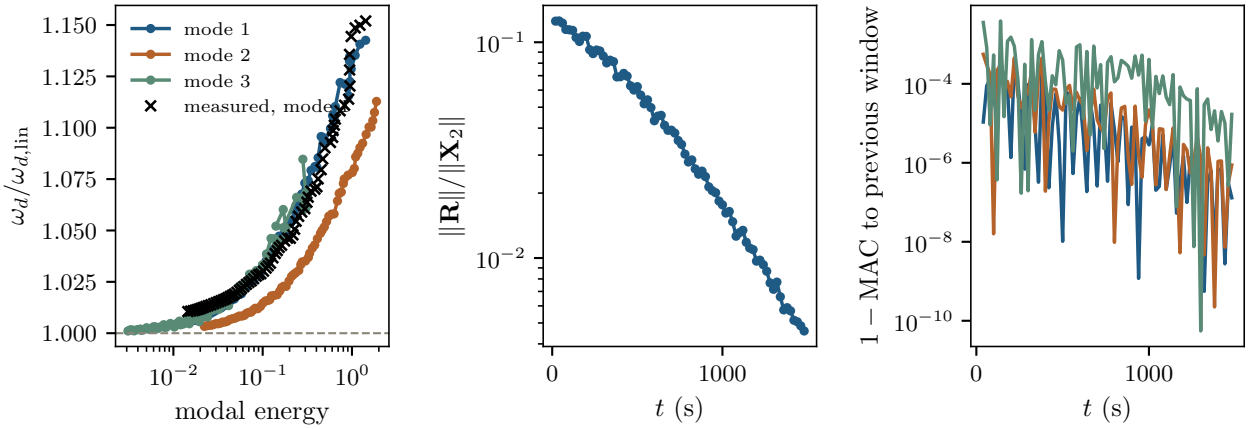


Figure 6 Example 5: 3-DOF chain with a cubic spring ($k_3 = 1$) grounding the first mass, damping $0.003\mathbf{M}$, one free decay of 1500 s, windows of 40 s every 20 s. Left: damped frequency against modal energy, normalised by the linear value; crosses are the measured response frequency of mode 1 in each window (largest difference 1.5 %). Centre: window residual. Right: shape change between consecutive windows.

4.2 Requirements and caveats

- **Excite every resonance you track.** A release along one mode makes the record rank deficient. Check the singular values `rec.s` of each window.
- **Each window must span at least one period of the slowest mode.** A dozen periods determine the pole well. Shorter windows resolve faster amplitude changes at the cost of scatter.
- **Weak modes lose their pole slot.** A window of a $2n$-dimensional record has $2n$ poles. If a resonance carries less energy than a superharmonic of a strong one, the superharmonic takes its slot. In our run of the Duffing chain of Example 5 with stiffness-proportional damping ($\mathbf{C} = 0.004\,\mathbf{K}$), mode 3 decayed early and its slot drifted to $3\omega_1$. Watch for a "mode" whose frequency approaches a multiple of another.
- **Validate every pole** against the measured response frequency (`band_frequency`); the comparison shows where the linear-pole description of a window holds.
- **Damping from single nonlinear windows scatters.** It can come out negative on a decaying record. Frequencies and shapes are the reliable outputs.

- **`center=True`** (the default in `track`) is needed whenever the motion is not about the origin.
- **The residual `tr.res`** is an indicator of how strongly each window engages the nonlinearity (and noise). It is not a calibrated measure.

4.3 Reduced-order-model diagnostics

The `rom` module works with the family of shapes one resonance takes over the swept energy range [2]: `robustness_basis` (SVD of the family), `deflate` (near-linear shape $\mathbf{v}_0$, leading amplitude correction and its relative size), `escape` (the part of each correction outside a retained span), `closure` (smallest retained set that contains the first k corrections) and `project` (Galerkin projection of $\mathbf{M}$, $\mathbf{K}$, $\mathbf{C}$ and the nonlinear force).

Remark. These quantities are built from differences of shapes. By Proposition 2, shapes from the superseded diagonal readout differ from the canonical shapes at first order in the off-diagonal part of $\mathbf{\Sigma}_2$. Values of corrections, escape and closure computed with the two readouts can therefore differ much more than the shapes do, and must not be mixed.

5 Reference

5.1 Installation

Clone `https://github.com/nld-uri/ncvd-toolbox`. Python 3.9 or later: `pip install -e python` (add `[plot,test]` for figures and tests); `python -m pytest -q` in `python/`. MATLAB R2022a or later: `addpath('.../matlab'); runtests('tests/test_ncvd.m')` in `matlab/`; `help ncvd` lists every function. Figures use LaTeX text: in Python `plotting.set_style()` (needs a TeX installation with `cm-super`), in MATLAB `ncvd.setStyle()`.

5.2 Functions

Python (`import ncvd as nc`)	MATLAB (`ncvd.*`)	Purpose
`ncvd(d, v, a, ...)`	`decompose(d, v, a, ...)`	canonical NCVD of a state record
`ncvd_pair(Y1, Y2, ...)`	`decomposePair(Y1, Y2, ...)`	any pair, $\mathbf{Y}_2 \approx \mathbf{Y}_1\mathbf{B}$
`balanced_quotient`	`balancedQuotient`	$\mathbf{Q}$
`regression_operator`	`regressionOperator`	LS, TLS, BCLS operator
`auto_time_scale`	`autoTimeScale`	time unit for an injective balanced map
`noise_variance`	`noiseVariance`	per-channel noise variance near Nyquist
`modal_parameters`	`modalParameters`	$\omega_n, \omega_d, \zeta, f_n, f_d$

Python (`import ncvd as nc`)	MATLAB (`ncvd.*`)	Purpose
`physical_modes`	`physicalModes`	one index per conjugate pair
`realify`	`realify`	real shape and MPC
`mac`, `mac_matrix`	`mac`, `macMatrix`	modal assurance criterion
`mac_phase_matched`	`macPhaseMatched`	MAC after Eq. (55) of [1]
`relative_error`	`relativeError`	Eq. (56) of [1]
`match_modes`	`matchModes`	one-to-one matching to reference modes
`deriv_filter`, `DerivFilter.band`	`derivFilter`, `filterBand`	FIR stencils and their bands
`state_from_displacement`	`stateFromDisplacement`	consistent $(\mathbf{q}, D\mathbf{q}, DD\mathbf{q})$
`correct_poles`	`correctPoles`	exact FIR pole correction
`track`, `window_edges`	`track`, `windowEdges`	sliding windows and tracking
`band_frequency`	`bandFrequency`	measured frequency for validation
`ncvd_ensemble`	`ensemble`	joint NCVD of several records
`rom.robustness_basis`, `rom.deflate`, `rom.escape`, `rom.closure`, `rom.project`	`robustnessBasis`, `deflate`, `escape`, `closure`, `project`	ROM diagnostics
`benchmarks.gcvd`, `.dmd`, `.itd`	`gcvd`, `dmd`, `itd`	published GCVD readout, DMD [7], ITD [8]
`systems.*`	`chain`, `paperChain10`, …	test systems, simulation, noise
`plotting.set_style`	`setStyle`	LaTeX text and palettes

5.3 Options of `ncvd` / `decompose`

Python / MATLAB	Default	Meaning
`rank` / `Rank`	none	whiten $\mathbf{Y}_1$ to this rank
`center` / `Center`	false	remove column means
`operator` / `Operator`	`'ls'`	`'ls'`, `'tls'`, `'bcls'`
`noise_var`, `fs` / `NoiseVar`, `Fs`	none	noise variances of $[\mathbf{q}, \dot{\mathbf{q}}]$ or `'auto'` (BCLS)
`forcing` / `Forcing`	none	measured input record, $m \times q$
`time_scale` / `TimeScale`	none	number c or `'auto'`
`weights` / `Weights`	none	row weights
`sort` / `Sort`	true	order by $\lvert \operatorname{Im} \lambda \rvert$
`structured` / `Structured`	true	keep the kinematic block exact in TLS/BCLS
`readout` / `Readout`	`'eig'`	`'diag'` gives the published readout (benchmarking)
`legacy_balanced` / `LegacyBalanced`	false	superseded shapes with assignment heuristic

5.4 Result fields

Field	Meaning
`lam`	characteristic values, λ_i paired with column i
`Phi`, `Psi`	regression shapes ($\mathbf{X}_1 = \mathbf{U}\mathbf{\Phi}^{\mathsf{H}}$) and eigenvector shapes $\mathbf{\Psi} = \overline{\mathbf{\Phi}}$
`U`	temporal basis
`R`	residual of the second identity (whitened coordinates if `rank`)
`res`, `exact1`	$\|\mathbf{R}\|/\|\mathbf{X}_2\|$ and $\|\mathbf{X}_1 - \mathbf{U}\mathbf{\Phi}^{\mathsf{H}}\|/\|\mathbf{X}_1\|$
`resid`	relative least-squares residual of $\mathbf{X}_2$ on $\mathbf{X}_1$
`V`, `T`, `S2`, `B`	balanced eigenvectors, rotation, $\mathbf{\Sigma}_2$, operator
`condT`, `condV`	condition numbers of $\mathbf{T}$ and $\mathbf{V}$
`s`	singular values of $\mathbf{X}_1$
`G`	input gains (with forcing)
`share`	pole-to-shape weight (1 except for the legacy readout)
`time_scale`, `operator`, `readout`	options used (MATLAB: `timeScale`)

6 Validation

- **Python, 28 tests**: reproduction of the original `ncvd.py` to 10^{-10} (values) and 10^{-8} (shapes, coordinates), with and without whitening and the legacy readout; identities and poles to 10^{-9} or better; Propositions 1 and 2; time-unit invariance; FIR correction for five stencils; forcing; ensembles; DMD and ITD on clean data; tracking on linear and Duffing records; BCLS bias reduction; ROM helpers.
- **MATLAB, 13 tests**, run on R2026b: the same checks, plus cross-checks against stored Python results: clean values to 10^{-10}, noisy values to 10^{-9} (LS), 10^{-8} (BCLS, diagonal readout, whitening) and 10^{-7} (TLS), the noise estimate to 10^{-8}, FIR correction, tracking to 10^{-8}, and the Duffing simulation to 10^{-6}.
- **Replication of [1]** (`python/examples/paper_replication.py`): the published and canonical readouts agree to 10^{-14} on the single-record noise study and the undersampling study. They differ by at most 0.5 % in individual poles on the ensemble study, with mode-averaged errors unchanged in trend.

7 Limitations

- The values are least-squares values (Proposition 1). Under random forcing with a finite record, the damping error is statistical, about 11–22 % in Example 3, and no readout removes it.
- BCLS assumes white sensor noise, independent between channels, and a noise-only band near Nyquist. Coloured noise, or noise from differentiating a noisy displacement, violates both assumptions.

- The FIR correction assumes the same stencil for both derivatives and holds only inside the stencil's band.
- Windowed values of nonlinear systems are equivalent-linear values of the window. Their meaning degrades with orbit anharmonicity and where another mode or a superharmonic dominates the window.
- Exact Joukowski ties (Section 3.5) affect only the published readout and the conditioning of $\mathbf{V}$, never the canonical values.

References

[1] H.-W.-X. Li, D. L. Stein, D. Chelidze, Characteristic value decomposition: A unifying paradigm for data-driven modal analysis, *Mechanical Systems and Signal Processing* 222 (2025) 111769, doi:10.1016/j.ymssp.2024.111769.

[2] D. L. Stein, D. Chelidze, Continuation of nonlinear normal modes using reduced-order models based on generalized characteristic value decomposition, *Nonlinear Dynamics* 113 (2025) 25–45, doi:10.1007/s11071-024-10239-0.

[3] B. F. Feeny, U. Farooq, A nonsymmetric state-variable decomposition for modal analysis, *Journal of Sound and Vibration* 310 (4) (2008) 792–800, doi:10.1016/j.jsv.2007.11.012.

[4] A. Savitzky, M. J. E. Golay, Smoothing and differentiation of data by simplified least squares procedures, *Analytical Chemistry* 36 (8) (1964) 1627–1639, doi:10.1021/ac60214a047.

[5] G. H. Golub, C. F. Van Loan, An analysis of the total least squares problem, *SIAM Journal on Numerical Analysis* 17 (6) (1980) 883–893, doi:10.1137/0717073.

[6] T. Söderström, Errors-in-variables methods in system identification, *Automatica* 43 (6) (2007) 939–958, doi:10.1016/j.automatica.2006.11.025.

[7] P. J. Schmid, Dynamic mode decomposition of numerical and experimental data, *Journal of Fluid Mechanics* 656 (2010) 5–28, doi:10.1017/S0022112010001217.

[8] S. R. Ibrahim, R. S. Pappa, Large modal survey testing using the Ibrahim time domain identification technique, *Journal of Spacecraft and Rockets* 19 (5) (1982) 459–465, doi:10.2514/3.62285.